LexIA

Plan du Projet

Juin 2026 · Mohamed Aouijil

Champ	Détail
Document	Plan du Projet — Version 1.1
Date	Juin 2026
Durée totale	8 semaines (2 mois)
Réalisateur	Mohamed Aouijil (projet individuel)
Méthodologie	Scrum simplifié — sprints hebdomadaires

1 Objectif du document

Ce plan définit l'organisation, le calendrier, les tâches, les jalons, les ressources et les risques identifiés pour le développement de LexIA. Il sert de référence tout au long des 8 semaines de développement.

Documents de référence

Document	Référence	Statut
Cahier des charges fonctionnel et technique	LexIA_CDC_Django.pdf	Validé
Plan du projet (ce document)	LexIA_Plan_Projet_v1.1.pdf	En cours
Maquettes UI / Wireframes	LexIA_Maquettes (Figma)	À produire
Documentation technique API	LexIA_API_Docs (DRF Swagger)	À produire

2 Réalisateur et rôles

Membre	Rôle	Responsabilités	Disponibilité
Mohamed Aouijil	Développeur Full-Stack & IA (seul réalisateur)	Backend : architecture Django · Modèles · API DRF · PostgreSQL · Tests backend IA & Données : corpus juridique · Pipeline RAG LangChain · Base vectorielle Frontend : interface React · TailwindCSS · Intégration API · UX/UI · Maquettes Figma · Tests frontend	Temps plein — 8 sem.

Organisation du travail en solo

- Chaque sprint traite d'abord le backend, puis le frontend qui le consomme, afin d'éviter les blocages.
- Auto-revue de code via Pull Requests GitHub avant chaque fusion sur la branche principale.
- Documentation technique et rapport final rédigés au fil des sprints.
- Bilan hebdomadaire écrit (30 min) : avancement, blocages, objectifs du sprint suivant.

3 Méthodologie de travail

Le projet suit une approche itérative inspirée de Scrum, adaptée à un développeur seul. Huit sprints d'une semaine, chacun avec un objectif clair et des livrables définis.

Pratique	Application dans LexIA
Sprint hebdomadaire	1 semaine = 1 sprint avec objectif et livrable définis
Point quotidien	10 min chaque matin : tâches du jour et blocages
Definition of Done	Code versionné sur GitHub + tests passants + fonctionnalité démontrée
Backlog de tâches	GitHub Issues / GitHub Projects avec priorités
Revue de sprint	Démonstration de la fonctionnalité en fin de semaine
Rétrospective	Bilan 15 min : ce qui a fonctionné / ce qu'on améliore

Outils de gestion

Outil	Usage	Justification
GitHub	Versionnage · Issues · Pull Requests	Standard industrie, historique complet
GitHub Projects	Backlog · Kanban · Suivi des tâches	Visualisation rapide de l'avancement
Figma	Maquettes UI · Wireframes · Design System	Conception visuelle avant développement
Postman	Test des endpoints API DRF	Validation des appels REST avant intégration frontend
VS Code	IDE principal (Django, React, Docker)	Environnement unique et configuré

4 Planning détaillé par sprint

Organisation du dépôt : **backend/** (Django, API, pipeline IA) · **frontend/** (interface web) · **data/** (corpus juridique).

S1 — Analyse & Conception

Backend & IA	Frontend	Livrables du sprint
› CDC · Architecture · Schéma BDD · ERD	› Wireframes Figma · Maquettes UI · Design System	✓ Cahier des charges finalisé ✓ Schéma ERD PostgreSQL ✓ Maquettes Figma validées ✓ Architecture technique documentée

Jalon 1 : Conception validée

S2 — Infrastructure & Setup

Backend & IA	Frontend	Livrables du sprint
› Projet Django init · PostgreSQL · Docker Compose · CI GitHub	› Projet React init · TailwindCSS · Structure composants · Routing	✓ Environnement Docker opérationnel ✓ Projet Django configuré ✓ Projet React configuré ✓ Pipeline CI GitHub Actions

Jalon 2 : Infrastructure prête

S3 — Backend Core — Modèles & API

Backend & IA	Frontend	Livrables du sprint
› Modèles Django · Migrations · Django Admin · Serializers DRF	› Composants chat UI · Layout principal · Axios config · Auth UI	✓ Modèles BDD migrés ✓ API CRUD fonctionnelle ✓ Django Admin configuré ✓ Composants React de base

S4 — Pipeline IA & Chatbot

Backend & IA	Frontend	Livrables du sprint
› LangChain · Chroma DB · Pipeline RAG · Vue Django /chat	› Interface chat complète · WebSocket ou polling · Affichage messages	✓ Pipeline RAG fonctionnel ✓ Endpoint /api/chat/ opérationnel ✓ Interface chat connectée au backend

Jalon 3 : Chatbot de base fonctionnel

S5 — Fonctionnalités avancées

Backend & IA	Frontend	Livrables du sprint
› Détection de catégorie · Support multilingue · Avertissements légaux	› Sélecteur de langue · Affichage RTL arabe · Suggestions de questions	✓ Classification automatique des questions ✓ Support FR/AR/Darija ✓ Avertissement légal affiché

S6 — Historique · Feedback · Admin Dashboard

Backend & IA	Frontend	Livrables du sprint
› Modèle Feedback · API historique · Statistiques admin · Export CSV	› Page historique · Composant feedback · Dashboard admin · Graphiques	✓ Système de feedback opérationnel ✓ Historique des conversations ✓ Dashboard administrateur

Jalon 4 : Application complète (v1)

S7 — Tests · Sécurité · Optimisation

Backend & IA	Frontend	Livrables du sprint
› Tests unitaires Pytest-Django · Audit sécurité JWT · Tests de charge	› Tests Vitest · Tests d'intégration · Accessibilité WCAG · Responsive fix	✓ Rapport de tests (couverture > 70%) ✓ Audit sécurité réalisé ✓ Bugs critiques corrigés

Jalon 5 : Tests validés

S8 — Finalisation & Livraison

Backend & IA	Frontend	Livrables du sprint
› Documentation API Swagger · README technique · Déploiement staging	› Rapport final · Slides présentation · Démo préparée	✓ Application déployée (Docker staging) ✓ Documentation complète ✓ Rapport final ✓ Présentation slides ✓ Démonstration live

Jalon 6 : Livraison finale

5 Diagramme de Gantt

Phase / Activité	S1	S2	S3	S4	S5	S6	S7	S8
Analyse & Conception								
Infrastructure & Setup								
Backend Core								
Pipeline IA & Chatbot								
Fonctionnalités avancées								
Feedback · Historique								
Tests & Sécurité								
Finalisation & Livraison								
Documentation (continu)								
Versionnage GitHub								

6 Jalons et points de contrôle

Jalon	Sprint	Critère de validation	Livrable	Statut
J1	S1	CDC finalisé, architecture validée, maquettes approuvées	CDC v1.1 · ERD · Maquettes Figma	À faire
J2	S2	Environnements backend et frontend opérationnels, Docker fonctionnel	Docker Compose · Projets initialisés	À faire
J3	S4	Chatbot répond via pipeline RAG, API /chat/ testée avec Postman	Pipeline RAG · Endpoint /api/chat/	À faire
J4	S6	Application v1 complète : chat + multilingue + feedback + admin	Application v1 · Dashboard admin	À faire
J5	S7	Couverture de tests > 70%, audit sécurité validé, 0 bug critique	Rapport de tests · Audit sécurité	À faire
J6	S8	Application déployée, documentation complète, démo prête	Docker staging · Rapport · Slides	À faire

7 Ressources techniques

Composant	Package / Outil	Version	Rôle
Backend	Django	5.2	Framework web principal
Backend	djangorestframework	3.16	API REST — Serializers · ViewSets · Routers
Backend	djangorestframework-simplejwt	5.5	Authentification JWT
Backend	django-cors-headers	4.9	Gestion des CORS pour React
Base de données	psycopg2-binary	2.9	Adaptateur PostgreSQL pour Django
IA	langchain	1.3	Orchestration du pipeline RAG
IA	langchain-openai	latest	Intégration LLM GPT-4 / GPT-3.5
IA	chromadb	1.5	Base vectorielle pour les embeddings
IA	sentence-transformers	latest	Génération d'embeddings locaux (fallback)
Frontend	react	18	Framework UI
Frontend	typescript	5	Typage statique
Frontend	tailwindcss	3	Design utilitaire CSS
Frontend	axios	1.18	Appels HTTP vers l'API DRF
Tests	pytest-django	4.11	Tests unitaires backend
Tests	factory-boy	3.3	Fixtures et données de test
DevOps	docker	26	Conteneurisation
DevOps	docker-compose	2	Orchestration multi-services

8 Analyse des risques

Risque	Prob.	Impact	Plan de mitigation
R1 — Intégration LangChain complexe / coût API LLM	Moyenne	Élevé	Fallback Mistral ou llama.cpp local · Prévoir budget API · Tester dès S4
R2 — Retard sur une fonctionnalité critique	Moyenne	Élevé	Priorisation MoSCoW · Réduire la portée d'une feature non critique plutôt que bloquer le jalon
R3 — Base de connaissances juridique insuffisante	Haute	Élevé	Alimenter la BDD dès S3 · Prévalider le contenu · Fallback par mots-clés
R4 — Difficultés NLP sur la Darija	Haute	Moyen	LLM multilingue (GPT-4 ou Mistral) · Limiter la Darija à la v2 si nécessaire
R5 — Charge de travail élevée (réalisateur unique)	Moyenne	Élevé	Backlog priorisé · Réutilisation de composants existants · Automatisation (CI, Docker) pour gagner du temps
R6 — Perte ou corruption de la base de données	Faible	Élevé	Sauvegardes PostgreSQL automatiques · Docker volumes persistants · Migrations versionnées
R7 — Manque de temps en fin de projet	Moyenne	Moyen	Buffer de 2 jours en S8 · Livrer une version réduite mais fonctionnelle
R8 — Failles de sécurité (injection, XSS)	Faible	Élevé	Protection native Django (CSRF/XSS) · DRF validation · Audit sécurité en S7

9 Tableau des livrables

	Livrable	Format	Responsable	Sprint
L1	Cahier des charges v1.1	DOCX / PDF	M. Aouijil	S1
L2	Schéma ERD de la base de données	PDF / PNG	M. Aouijil	S1
L3	Maquettes UI et wireframes	Figma / PDF	M. Aouijil	S1
L4	Environnement Docker Compose	docker-compose.yml	M. Aouijil	S2
L5	Projet Django initialisé	GitHub Repository	M. Aouijil	S2
L6	Projet React + TailwindCSS initialisé	GitHub Repository	M. Aouijil	S2
L7	Modèles Django + migrations PostgreSQL	Code Python	M. Aouijil	S3
L8	API CRUD Django REST Framework	Endpoints DRF	M. Aouijil	S3
L9	Interface React — composants de base	TypeScript/JSX	M. Aouijil	S3
L10	Pipeline RAG LangChain + Chroma DB	Module Python	M. Aouijil	S4
L11	Interface chatbot connectée au backend	React + Axios	M. Aouijil	S4
L12	Support multilingue FR/AR/Darija	Frontend + Backend	M. Aouijil	S5
L13	Système de feedback utilisateur	API + Interface	M. Aouijil	S6
L14	Dashboard administrateur	Django Admin + React	M. Aouijil	S6
L15	Rapport de tests (couverture > 70%)	PDF / HTML	M. Aouijil	S7
L16	Audit sécurité (JWT, CSRF, XSS)	Document PDF	M. Aouijil	S7
L17	Application déployée (Docker staging)	URL + Docker	M. Aouijil	S8
L18	Documentation API (DRF Swagger)	URL / PDF	M. Aouijil	S8
L19	Rapport technique final	PDF	M. Aouijil	S8
L20	Présentation slides	PDF / PPTX	M. Aouijil	S8
L21	Démonstration live	Session présentielle	M. Aouijil	S8

10 Critères de réussite

Critère	Indicateur mesurable	Priorité
CR1 — Application accessible via navigateur	HTTP 200 sur la page d'accueil	Haute
CR2 — Réponses pertinentes aux questions	Taux de pertinence > 80% sur 20 questions de test	Haute
CR3 — Pipeline RAG sans hallucination majeure	0 hallucination majeure sur les questions testées	Haute
CR4 — Support FR, arabe et darija	Test dans les 3 langues avec affichage RTL correct	Haute
CR5 — Avertissement légal affiché	Présent dans 100% des réponses testées	Haute
CR6 — CRUD administrateur fonctionnel	CRUD fonctionnel sur catégories, Q&R et utilisateurs	Haute
CR7 — Couverture de tests > 70%	Rapport pytest-cov > 70%	Moyenne
CR8 — Temps de réponse API < 3 secondes	Mesuré avec Postman sur 10 requêtes consécutives	Moyenne
CR9 — Déploiement Docker en une commande	docker-compose up sans erreur	Haute
CR10 — Documentation complète	README · Swagger · Rapport final présents et cohérents	Moyenne

11 Conclusion

Ce plan couvre 8 sprints, 6 jalons et 21 livrables, tous réalisés par Mohamed Aouijil. Le dépôt est séparé en trois parties — backend (API et IA), frontend (interface) et data (corpus juridique) — ce qui permet d'avancer couche par couche et de garder un rythme de livraison régulier.

Les 8 risques identifiés ont chacun un plan de mitigation concret, y compris la charge de travail propre à un projet individuel. Les critères de réussite sont mesurables et couvrent les dimensions fonctionnelles, techniques et qualitatives du projet.

La stack Django 5 + DRF + React 18 + PostgreSQL + LangChain + Docker reste la colonne vertébrale technique, cohérente avec le cahier des charges v1.1.

> *LexIA — Plan du Projet · Mohamed Aouijil · 8 Sprints · 6 Jalons · 21 Livrables · 8 Risques · Django 5 · DRF · React 18 · PostgreSQL · LangChain · Docker*